

Deep Dive: Distributed Systems

Trade-offs & Document Data Modeling

The transition from a purely relational database paradigm—rooted in normalization, foreign key constraints, and vertical scaling—to the realm of distributed NoSQL systems requires a fundamental shift in architectural thinking. For students with a strong background in PostgreSQL, DuckDB, and the principles of the Third Normal Form (3NF), the theoretical guarantees of local Online Transaction Processing (OLTP) systems often feel like immutable laws of computing. However, the introduction of distributed nodes communicating over fallible networks, coupled with the flexible nature of document-oriented schemas, introduces novel theoretical constraints and intricate design patterns.

This comprehensive reading document explores the trade-offs inherent in distributed computing architecture and provides an exhaustive examination of advanced data modeling techniques required to optimize document stores like MongoDB. By bridging your existing knowledge of relational structures with the realities of distributed networks, this guide will equip you to make informed architectural decisions.

Part 1: The CAP Theorem in Depth

To master distributed databases, one must first comprehend the absolute theoretical boundaries that govern them. The CAP theorem serves as the foundational framework for evaluating how distributed systems behave under conditions of failure.

The Brewer Conjecture and the Gilbert-Lynch Proof

The concept of CAP originated as a conjecture presented by Dr. Eric Brewer during his keynote address at the 2000 ACM Symposium on Principles of Distributed Computing (PODC).¹ During a period when the internet was scaling rapidly, Brewer posited that any distributed shared-data system could provide at most two of three desirable properties simultaneously:

1. **Consistency (C):** Every read receives the most recent write or an error. In this strict definition, all clients see the exact same data simultaneously, regardless of the specific node they connect to.¹
2. **Availability (A):** Every request received by a non-failing node must result in a response, without the absolute guarantee that the response contains the most recent version of the data.¹
3. **Partition Tolerance (P):** The system continues to operate despite an arbitrary number of messages being dropped or delayed by the network between nodes.¹

Brewer's conjecture was based on practical observations from building giant-scale systems, but it lacked mathematical rigor at the time.² In 2002, researchers Seth Gilbert and Nancy

Lynch of MIT published a formal proof of Brewer's conjecture, officially elevating it to the status of a theorem.¹

Gilbert and Lynch utilized the asynchronous network model for their proof.⁴ In an asynchronous network, there is no global clock, meaning nodes cannot rely on synchronized timestamps to perfectly order events, and they must make decisions based solely on local computation and the messages they successfully receive.⁵ Their proof demonstrated by contradiction that if a network is divided into two disjoint sets of nodes that cannot communicate (a partition), a write to one side of the partition fundamentally cannot be replicated to the other.⁵ If a subsequent read occurs on the isolated side, the system is trapped in a dilemma: it must either return a stale, outdated value (thereby sacrificing Consistency) or it must refuse to respond to the client (thereby sacrificing Availability).⁵

The Oversimplification of "Pick 2 of 3"

A pervasive and enduring misconception surrounding the CAP theorem is the "pick 2 of 3" paradigm.³ This interpretation suggests that software architects can freely select between building a CA (Consistent and Available), CP (Consistent and Partition Tolerant), or AP (Available and Partition Tolerant) system.³

This interpretation implies that Partition Tolerance is an optional feature that one can simply "opt-out" of. In reality, network partitions are inevitable physical realities in distributed computing.³ Hardware routers fail, network cables are severed, and heavy traffic causes packet loss. Because a distributed system is deployed over a network, the system has no choice but to be Partition Tolerant.³

The true interpretation of the CAP theorem, as clarified by Brewer himself twelve years later, is not a simple choice of two out of three.¹ Instead, the real question is: **"Given that a network partition will inevitably occur, does your system choose to sacrifice Consistency or Availability?"**³ When a partition happens, an AP system will continue answering queries with potentially stale data to keep the application online, whereas a CP system will proactively shut down or block queries to the isolated nodes to prevent inconsistent reads, causing an outage from the perspective of the client.⁹

Concrete Walk-through 1: 3-Node Replicated Database Under Partition

To fully grasp the difference between CP and AP behavior, we can trace a step-by-step scenario involving a standard 3-node distributed database cluster experiencing a network failure.

System State Definition:

- **Node 1:** The designated Leader node, responsible for accepting write operations and

serving read operations.

- **Node 2:** A Follower node, which accepts read operations and continuously replicates data from Node 1.
- **Node 3:** A Follower node, identically configured to accept read operations and replicate from Node 1.
- **Data State:** A specific user profile variable, `account_status`, is currently set to active across all three nodes.

The Step-by-Step Scenario:

1. **The Network Partition Occurs:** A physical network switch fails within the data center. Node 1 and Node 2 remain connected and can communicate with each other. However, Node 3 is completely severed from the other two nodes. The network is now in a partitioned state.
2. **The Write Operation:** Client A connects to the Leader (Node 1) and issues a write request to update `account_status` to suspended.
3. **The Replication Attempt:** Node 1 updates its local storage to suspended and successfully replicates this state change to Node 2. Both Node 1 and Node 2 now hold the suspended state. Node 1 then attempts to replicate the update to Node 3, but the network drops the message due to the partition.⁶
4. **The Read Operation:** Client B connects to the isolated Node 3 and issues a read request for the user's `account_status`. Node 3 is unaware of the write that occurred on Node 1.
5. **The System's Choice (CP vs. AP):** Node 3 must now make a deterministic choice based on its configuration:
 - **AP Behavior (Availability Prioritized):** Node 3 is programmed to prioritize responding to the client above all else. It returns its current, local value to Client B, which is active. The system remains highly available, satisfying the client's request. However, Consistency is broken because Client B received a stale state, entirely missing Client A's recent write.⁶
 - **CP Behavior (Consistency Prioritized):** Node 3 is programmed to know that it has lost its heartbeat connection with the Leader. To prevent serving potentially stale data, Node 3 refuses the read request from Client B, either by returning an explicit 503 Service Unavailable error or by allowing the request to time out. Consistency is perfectly maintained because no client sees conflicting data, but Availability is sacrificed because Client B's request was rejected.⁶

Martin Kleppmann's Critique: Stopping the "CP" vs "AP" False Dichotomy

While the CAP theorem is an essential educational tool, prominent distributed systems researcher Martin Kleppmann argues forcefully that classifying modern databases strictly as "CP" or "AP" is deeply flawed and should be abandoned by engineers.⁸ Kleppmann highlights several critical reasons why the CAP definitions are too rigid to describe the nuances of

contemporary distributed architectures.

First, Kleppmann notes that CAP defines "Consistency" in a highly specific, mathematical way known as *linearizability*.⁸ Linearizability is the illusion that there is only one copy of the data in the entire system, and every operation takes effect instantaneously.⁸ This is entirely different from the "C" in ACID, which refers to preserving business rules like foreign keys and uniqueness constraints.¹ Achieving true linearizability is incredibly expensive regarding performance; even modern multi-core CPUs do not provide linearizable access to their own local RAM without developers utilizing explicit, costly memory barrier instructions.⁸

Second, Kleppmann demonstrates that many popular distributed systems fit neither the CP nor the AP definition.⁸ Consider standard single-leader replication, which is widely used in relational systems like PostgreSQL. These systems are not CAP-Available because if a client is partitioned from the leader node, it cannot perform a write operation.⁸ Conversely, if they utilize asynchronous replication (where the leader does not wait for followers to acknowledge writes), they are not CAP-Consistent, because a client reading from a lagging follower will see stale, non-linearizable data.⁸

Kleppmann points to Apache ZooKeeper as a prime example of this categorization failure. ZooKeeper is universally branded as a "CP" system. Yet, by default, a client reading from a ZooKeeper node sees the data specifically on that local node, which may be lagging behind the leader.⁸ To get a mathematically linearizable (CAP-Consistent) read, a user must explicitly invoke a sync command.⁸ Kleppmann asserts that reducing the complex, configurable fault-tolerance spectrum of a database to a single binary "CP/AP" label obscures vital operational realities and leads to poor architectural decisions.⁸

Part 2: The PACELC Theorem

Recognizing the practical limitations of the CAP theorem's binary constraints, Dr. Daniel Abadi introduced the PACELC theorem in 2012.¹¹ PACELC addresses a critical, everyday gap in the CAP framework: what trade-offs must a database make when the system is operating normally and there is no network partition?

Daniel Abadi's Extension

PACELC is an acronym that elegantly expands the design space. It states: **If Partition (P), choose Availability (A) or Consistency (C); Else (E), choose Latency (L) or Consistency (C).**¹

Abadi realized that network partitions, while inevitable over a long enough timeline, are relatively rare events.⁷ For the vast majority of its operational life, a distributed system exists in a state without partitions. Yet, even in this healthy state, a fundamental, inescapable trade-off

exists between latency and consistency.¹¹

If a database architecture guarantees strong consistency across a cluster of nodes, every single write operation must be synchronously acknowledged by a majority of those nodes before the database can confirm the write's success to the user.¹¹ This requirement for a full network round-trip between physical machines inherently increases the latency of the operation.¹¹ Conversely, if the system is designed to acknowledge the write immediately after updating the local node's memory (asynchronous replication), it provides lightning-fast low latency to the user, but it risks returning inconsistent, stale data to concurrent readers on other nodes.¹¹

PACELC is significantly more practical than CAP for system selection because it forces database architects to evaluate the system's day-to-day performance overhead and user experience, rather than merely planning for catastrophic network failure states.⁷

System Classification under PACELC

Using the PACELC framework, distributed data stores can be classified into specific architectural combinations: PA/EL, PC/EC, PA/EC, and PC/EL.

Table 1: PACELC Classifications of Modern Databases

Database System	PACELC Classification	Architectural Rationale
Amazon DynamoDB	PA/EL	Designed for massive scale. Prioritizes availability during network failures. Normally prioritizes extremely low latency via eventual consistency, allowing replicas to sync asynchronously. ¹³
Apache Cassandra	PA/EL	Highly available during partitions. Built for high-velocity write workloads, choosing low latency over strict consistency during normal operations. ¹⁵

Redis Cluster	PA/EL	In its default cluster configuration, Redis focuses on microsecond latency and remains available during splits. It accepts eventual consistency, avoiding complex merge operations that would block the main thread. ¹⁶
MongoDB	PA/EC	(Default Configuration). It remains highly available during partitions by dynamically electing a new primary node. However, during normal operations, reads and writes default to the primary, prioritizing strong consistency (EC) over localized low latency. ¹⁶
CockroachDB	PC/EC	A distributed SQL system implementing the Raft consensus algorithm. It strictly prioritizes strong consistency at all times, accepting both unavailability during partitions and higher latency during normal operations to guarantee flawless transactional integrity. ¹⁴
PostgreSQL	PC/EC	A fully ACID-compliant relational system. It will not yield consistency under any circumstance, accepting the necessary latency overhead for MVCC and availability limits to maintain

		relational constraints. ¹¹
--	--	---------------------------------------

Part 3: Consistency Models Spectrum

For engineers coming from relational environments, the term "consistency" usually means that a transaction will not violate a database's defined rules (like a foreign key constraint). In distributed systems theory, consistency refers to the chronological visibility of data across multiple nodes. This is not a binary switch, but rather a spectrum of models that dictate the timing of data propagation.

Strong to Weak Consistency Models

Understanding the hierarchy of consistency models allows data engineers to select the appropriate guarantee for a specific operational workload, balancing the need for correctness against the cost of coordination.

1. Linearizability (Strong Consistency) Linearizability is the strongest model, acting as the strict "C" in the CAP theorem.⁸ It guarantees that once a write operation completes successfully, all subsequent read operations by any client, on any node, must return the new value.¹⁰ From the perspective of the application, operations appear to execute instantaneously in a global, real-time order.¹⁰

The cost of linearizability is extreme coordination overhead.²² To achieve this, distributed systems must implement rigorous consensus algorithms (such as Raft or Paxos) that require continuous network communication to establish a total order of events.²³ This drastically increases latency and limits the scalability of the system, as nodes must constantly check with leaders or majorities before responding to client requests.²⁴

2. Sequential Consistency Slightly weaker than linearizability, sequential consistency ensures that operations occur in a global order that aligns with the program order of each individual process, but it relaxes the requirement for strict real-time ordering across all processes simultaneously.²⁴ It guarantees that all clients will see the exact same sequence of events, ensuring logical correctness, but that sequence might drift slightly from absolute atomic real-time.²⁴ This subtle relaxation allows for more efficient implementations than strict linearizability, though it still incurs significant coordination costs.²⁷

3. Causal Consistency Causal consistency tracks the logical dependencies between operations. It guarantees that operations that are causally related are seen by every node in the correct order.²⁶ For example, if User A posts a comment (Write 1) and User B replies to that comment (Write 2), causal consistency guarantees that no client in the world will ever see the reply (Write 2) without first seeing the original comment (Write 1).²⁸ However, concurrent events with no causal relationship (e.g., User C posting a completely unrelated comment) can be observed in any order by different clients. This model strikes a powerful balance, preventing

logical absurdities while avoiding the massive latency penalty of linearizability.

4. Eventual Consistency This is the weakest but most highly performant model. Eventual consistency guarantees that if no new updates are made to a given data item, eventually all accesses to that item will converge to return the last updated value.⁹ This model allows nodes to absorb writes instantly and replicate them asynchronously in the background.⁹ The real-world latency—the time window during which nodes might serve inconsistent data—is typically measured in milliseconds, but during heavy loads or network partitions, it can stretch to seconds or longer. It offers incredibly low latency but pushes the burden of handling read-write anomalies (like reading a stale balance on an account) onto the application developer.⁹

Session Guarantees in Document Stores

To bridge the gap between strong and eventual consistency without paying the full penalty of linearizability, document stores often implement "session guarantees." A session represents an individual client's logical connection to the database over time.²⁹

- **Read-your-writes:** A client will immediately see the data it just updated within its own session, even if other clients across the network do not yet see it.²⁸
- **Monotonic reads:** If a client reads a specific, updated version of data, any subsequent reads by that client in the same session will never return a chronologically older version of that data.²⁸

Configurable Consistency via MongoDB's Read and Write Concerns

Relational databases typically enforce consistency globally at the database level. MongoDB, uniquely, handles the consistency spectrum by allowing developers to configure isolation levels on a per-query or per-session basis using readConcern and writeConcern parameters.³⁰

- **Write Concern (w):** This dictates how many nodes must acknowledge a write before the MongoDB driver considers it successful. A setting of {w: 1} means only the Primary node acknowledges it in memory. This yields exceptionally low latency but risks data loss if the Primary crashes before replicating.²⁸ Conversely, a setting of {w: "majority"} forces the database to wait until a mathematically calculated majority of data-bearing nodes have securely written the data to their oplogs (operations logs).²⁸
- **Read Concern:** This determines the isolation level of read operations. A "local" concern reads the most recent data present on the specific node, but this data might be rolled back if a Primary fails.²⁸ A "majority" concern guarantees that the data returned to the client has been durably committed by a majority of nodes and is completely safe from rollback.²⁸

When developers purposefully combine {w: "majority"} and readConcern: "majority" within a

client session, MongoDB automatically enables full **Causal Consistency**.²⁸

Table 2: MongoDB Causal Consistency Guarantees Based on Concerns²⁸

Read Concern	Write Concern	Read Your Writes	Monotonic Reads	Monotonic Writes	Writes Follow Reads
"majority"	"majority"	Guaranteed	Guaranteed	Guaranteed	Guaranteed
"majority"	{ w: 1 }	Not Guaranteed	Guaranteed	Not Guaranteed	Guaranteed
"local"	{ w: 1 }	Not Guaranteed	Not Guaranteed	Not Guaranteed	Not Guaranteed

Concrete Walk-through 2: Causal Consistency Session

Let us trace how MongoDB provides causal consistency under the hood using operation and cluster timestamps, demonstrating why {w: 1} fails to provide a "Read Your Writes" guarantee under certain failure modes.

The Scenario:

1. **Session Initiation:** A client application starts a causally consistent session. MongoDB assigns logical timestamps to track the order of operations.²⁸
2. **The Write:** The client issues an update to an inventory document, reducing the stock of "Laptops" from 5 to 4. The client uses a weak write concern: {w: 1}.
3. **Local Acknowledgment:** The Primary node (Node P) applies the write to its local memory and immediately acknowledges success to the client. The client's session logical time is advanced.
4. **Network Failure:** Before Node P can replicate this write to the Secondary nodes (Node S1 and S2), Node P experiences a sudden hardware crash.
5. **Failover:** Node S1 is elected as the new Primary. Node S1 never received the update, so its inventory still shows 5 Laptops.
6. **The Read Request:** Milliseconds later, the same client issues a read request for the inventory document, utilizing readConcern: "majority". Because it is in a causally consistent session, the client sends its logical timestamp, demanding to see a state of the database

that includes its recent write.

7. **The Guarantee Failure:** The new Primary (Node S1) looks at the client's logical timestamp and realizes it does not have the data corresponding to that time (because the write was lost when Node P crashed). Therefore, it cannot fulfill the "Read Your Writes" guarantee.
8. **The Solution:** If the client had utilized {w: "majority"} in Step 2, the original Primary would have been forced to replicate the inventory reduction to Node S1 before acknowledging success. The causal consistency guarantee would have been upheld perfectly despite the hardware crash, because Node S1 would possess the updated data.²⁸

Part 4: Document Data Modeling in Depth

In relational databases, data modeling is dictated by normalization algorithms designed to eliminate redundancy. In NoSQL document modeling, the schema is dictated entirely by *application access patterns*. The golden rule of document stores is: **Data that is accessed together should be stored together.**

The 16 MB Limit and its Architectural Implications

A defining characteristic of MongoDB is that it enforces a strict 16 MB maximum size limit per BSON (Binary JSON) document.³¹ For developers accustomed to databases that allow massive TEXT or BLOB columns, this can seem restrictive. However, this is not an arbitrary limitation; it is an architectural safeguard.³¹

In a document database, a single document is the smallest unit of retrieval. When a query matches a document, the entire document must be read from disk into RAM, processed by the CPU, and transmitted over the network to the client application.³² If documents were allowed to grow indefinitely—for instance, a document representing a user profile that also embeds every single action that user has ever taken on a platform—a single query could easily pull gigabytes of data into memory.³¹ This would cause sudden memory evictions, choke network bandwidth, and lead to catastrophic performance degradation for the entire cluster.³¹ The 16 MB limit forces developers to design scalable schemas and aggressively penalizes anti-patterns like unbounded arrays.³³

Six Essential MongoDB Data Modeling Patterns

To navigate the 16 MB limit and optimize read/write latency, MongoDB schema design relies on specific, well-documented patterns.³⁴

1. The Attribute Pattern

- **The Problem:** Large documents often contain many similar fields, or fields that are sparse (only present in a small percentage of documents).³⁵ Creating traditional single-field indexes for every possible sparse field wastes vast amounts of RAM and slows down write operations.³⁵

- **The Solution:** Move these fields into an array of key-value pairs.³⁵
- **Concrete Example:** Consider an e-commerce catalog featuring clothing, electronics, and food. Electronics have "voltage", clothing has "size", and food has "calories". Instead of creating dozens of top-level fields, structure the document using the Attribute Pattern: { "specs": [{ "k": "voltage", "v": "120V" }, { "k": "size", "v": "Large" }] }.³⁵ This allows the database administrator to create a single compound multikey index on specs.k and specs.v that covers every possible product attribute efficiently.³⁵

2. The Bucket Pattern

- **The Problem:** Storing time-series data or IoT sensor streams as individual documents (e.g., one document per sensor reading per minute) creates billions of documents, severely bloating the index size and consuming immense storage overhead.³⁶
- **The Solution:** Group multiple sequential readings over a specific timeframe into a single "bucket" document.³⁶
- **Concrete Example:** A temperature sensor taking readings every minute. Instead of 60 individual documents per hour, the system generates one document representing the 1-hour time span. This document contains start_time and end_time fields, an array containing 60 sub-documents of timestamps and temperatures, and pre-aggregated totals like sum_temperature and transaction_count.³⁶ This drastically reduces the total number of documents in the collection and shrinks the B-Tree index footprint.³⁶

3. The Computed Pattern

- **The Problem:** In highly read-intensive applications, calculating aggregates on the fly for every single read request consumes excessive CPU cycles and disk I/O, leading to bottlenecks.³⁷
- **The Solution:** Pre-compute the results of aggregations during write operations and store them directly in the document, shifting the compute burden from read time to write time.³⁷
- **Concrete Example:** A movie database website displaying the total box office revenue for a film. Instead of running an expensive \$sum aggregation across millions of individual ticket sale documents every time a user loads the movie page, the system utilizes the Computed Pattern. Whenever a batch of tickets is sold, an asynchronous background process recalculates the revenue and updates a persistent total_revenue field directly on the main Movie document.³⁸

4. The Extended Reference Pattern

- **The Problem:** When dealing with many-to-one (N-1) relationships, relying heavily on \$lookup (the MongoDB equivalent of a JOIN) severely impacts read performance. Conversely, fully embedding the entire related document leads to massive data duplication and synchronization nightmares.³⁷
- **The Solution:** Embed only the highest-priority, most frequently accessed fields into the

primary document, leaving the less important historical data in a separate referenced collection.³⁹

- **Concrete Example:** Order management. A Customer document contains a name, shipping_address, secondary_phone, tax_id, and billing_history.³⁹ When that customer creates an Order, the system copies *only* the customer_id, name, and shipping_address directly into the Order document.³⁹ The warehouse can now process the order with a single fast read, without executing a JOIN.³⁹ Furthermore, this acts as a historical snapshot; if the customer changes their address next year, the old order accurately reflects where it was originally shipped.³⁹

5. The Outlier Pattern

- **The Problem:** A small percentage of documents in a collection do not fit the normal data pattern and threaten to exceed the 16 MB limit or distort query performance for the rest of the collection.³⁶
- **The Solution:** Optimize the schema for the 99% of typical documents. For the 1% of outliers, add a flag and overflow the excessive data into a separate, linked document.³⁶
- **Concrete Example:** A social network storing a list of followers inside a user's profile document. For standard users, an array of 500 follower IDs is perfectly efficient. However, a celebrity account might have 50 million followers, which will immediately break the 16 MB limit.³⁶ The system detects this, sets a flag has_extras: true on the celebrity's document, limits the embedded array to 1,000 recent followers, and stores the remaining 49,999,000 followers in separate linked "overflow" documents.³⁶ Application logic detects the flag and routes queries accordingly.³⁶

6. The Schema Versioning Pattern

- **The Problem:** Relational databases require ALTER TABLE commands that lock tables and demand massive downtime to migrate schemas. In agile development, schemas must evolve constantly.³⁷
- **The Solution:** Add a schema_version integer field to all documents. The application code detects the version and handles the parsing seamlessly, allowing older document structures to coexist safely with newer ones.⁴⁰
- **Concrete Example:** A user profile collection initially stores contact info as flat fields: home_phone and work_phone (Version 1). A new business requirement dictates that users can have infinite social media links. The schema is updated to use an array of objects called contact_methods (Version 2). By checking the schema_version field, the application can read both structures. When a Version 1 user logs in and updates their profile, the application silently migrates their data to the Version 2 structure upon writing.⁴¹

Anti-Patterns in Document Stores

Students transitioning from relational databases often map old habits onto document stores,

resulting in severe performance anti-patterns:

- **Unnecessary Normalization:** Splitting data across multiple collections purely to mimic 3NF. In MongoDB, this forces the database to rely heavily on \$lookup operations. Unlike relational engines optimized for complex JOINS in memory, document JOINS are comparatively slow and degrade read performance.³⁹
- **Massive Unbounded Arrays:** Embedding continuously growing lists—such as server logs or every comment on a viral video—into a single document. This inevitably triggers the 16 MB limit and causes fragmentation on the storage engine as the document frequently changes size.³³
- **Deep Nesting:** MongoDB technically permits up to 100 levels of document nesting.⁴³ However, deeply nested arrays of arrays complicate the application code required to parse them and make efficient indexing nearly impossible.⁴³
- **Storing Large BLOBs in Documents:** Attempting to store large images or video files as base64 strings inside standard BSON documents. These should either be stored in cloud object storage (like AWS S3) with a URL reference in the document, or managed using MongoDB's GridFS API, which automatically chunks large files into smaller manageable documents.³²

Modeling Time-Series Data: Native vs. Bucket Pattern

While the Bucket Pattern is an effective manual technique for modeling time-series data, it requires significant development overhead to write the aggregation and bucketing logic in the application tier.⁴⁴

To solve this, MongoDB 5.0 introduced native Time-Series Collections.³⁶ In standard collections, documents are stored sequentially on disk, requiring a massive B-Tree index for every data point.⁴⁵ Native time-series collections utilize an advanced internal columnar format.⁴⁵ Developers simply insert individual data points, and the database engine automatically groups documents behind the scenes based on a defined metaField (such as a sensor ID) and time granularity.⁴⁵ This hybrid approach offers the best of both worlds: the developer interface is simple (inserting one document at a time), but the internal storage density and index size match the highly optimized footprint of the Bucket Pattern, resulting in dramatically reduced I/O for read operations.⁴⁴

Index Considerations: Embedded vs. Referenced

In relational databases, indexes target simple scalar columns. In MongoDB, indexing requires strategic awareness of BSON structures.

If you choose to use the Extended Reference Pattern, you index a simple scalar field (like `customer_id`), which behaves exactly like a B-Tree index in PostgreSQL. However, if you embed an array of data, MongoDB automatically creates a **multikey index**.⁴⁷ If an array contains 100 elements, the database engine must extract every single element and create 100 distinct index

pointers mapping back to that single document.⁴⁷ While multikey indexes brilliantly accelerate read queries across complex arrays, they impose a severe CPU and RAM overhead during write operations, as a single document insert requires 100 index updates.⁴⁷ Therefore, embedding arrays is powerful, but must be paired with patterns (like Attribute or Outlier) to ensure the index does not become a write bottleneck.³⁵

Part 5: MongoDB vs. PostgreSQL JSONB

Modern architectural decisions rarely present a clean binary between "SQL" and "NoSQL." A significant convergence trend has blurred the rigid lines separating these database paradigms.⁴⁷

PostgreSQL JSONB as the Middle Ground

PostgreSQL introduced the JSONB column type to handle semi-structured data while remaining firmly within a relational framework.⁴⁷ Unlike standard JSON types which store data as raw text strings requiring parsing on every read, JSONB stores the data in a decomposed, binary format.⁴⁷

This creates a powerful middle ground for data modeling. An architect can maintain a rigid, highly normalized relational schema for core business logic (e.g., users, authentication, and financial ledgers) while simultaneously utilizing a flexible JSONB column to store highly variable data (e.g., product specifications, dynamic form responses, or user-defined UI settings) within the exact same row.⁴⁷

Indexing Capabilities: GIN vs. MongoDB Multikey

The mechanism by which these two databases index and search unstructured data highlights their architectural differences:

- **PostgreSQL GIN (Generalized Inverted Index):** PostgreSQL relies heavily on GIN indexes to query JSONB columns.⁴⁷ An inverted index acts much like the index at the back of a textbook; it maps every distinct key and value found anywhere within the JSON structure back to the specific row ID.⁴⁷ This makes GIN indexes exceptionally fast for containment queries (using the `@>` operator to ask, "Does this row's JSON contain `{\"status\": \"active\"}?`").⁴⁷ However, GIN indexes are notoriously expensive to update, meaning high-velocity write workloads can experience performance degradation.⁴⁹
- **MongoDB Indexes:** Because MongoDB was built natively on BSON, indexing nested document fields is not an add-on feature but the core architecture of the engine.⁴⁷ MongoDB relies on traditional B-Tree-backed single-field indexes, multikey indexes for arrays, and supports dynamic wildcard indexing when the exact schema topology is completely unknown ahead of time.⁴⁷ There is no "impedance mismatch" between the application's JSON object and the database's index capabilities.⁴⁷

Transaction Support and The Convergence Trend

Historically, the defining advantage of PostgreSQL was its monopoly on full ACID (Atomicity, Consistency, Isolation, Durability) transactions. PostgreSQL manages this via Multi-Version Concurrency Control (MVCC), which ensures that readers and writers do not block each other, allowing massive, long-running analytical queries (OLAP) to execute safely alongside continuous transactional writes (OLTP).⁴⁷ In its early years, MongoDB deliberately lacked multi-document transactions to prioritize extreme horizontal write scaling, guaranteeing atomicity only at the single-document level.⁴⁷

However, the platforms are actively converging upon each other's territory.⁴⁷ MongoDB officially introduced multi-document ACID transactions in version 4.0, moving closer to relational safety guarantees.⁴⁷ (It must be noted that these NoSQL transactions incur a performance overhead, carry a default 60-second timeout, and remain optimized for operations within a single replica set ⁴⁷). Concurrently, PostgreSQL has absorbed major NoSQL capabilities, implementing robust jsonpath support (introduced in version 12) for complex document querying.⁴⁷ Furthermore, MongoDB has added relational features, such as \$lookup for cross-collection joins and strict schema validation rules.⁴⁷

Table 3: PostgreSQL JSONB vs. MongoDB Feature Comparison

Architectural Feature	PostgreSQL (using JSONB)	MongoDB (BSON)
Native Storage Format	Relational rows containing specific decomposed Binary JSON (JSONB) columns.	Pure Binary JSON (BSON) collections and documents from the ground up.
Indexing Strategy	GIN (Generalized Inverted Index) for efficient JSON containment, B-Tree for scalar columns.	Native B-Tree, Multikey for arrays, Wildcard indexes for completely dynamic schemas.
Transaction Guarantees	Full, unlimited ACID transactions via MVCC. No duration limits.	Multi-document ACID available (since v4.0), but optimized primarily for single-document atomicity.

Scaling Architecture	Vertically scaling primary. Horizontal sharding exists but is complex and requires third-party tools.	Built natively for horizontal scaling via automated sharding across distributed clusters.
Query Flexibility	ANSI SQL combined with jsonpath operators for document traversal.	Native MongoDB Query API (JSON-like objects interacting via drivers).

When choosing between PostgreSQL JSONB and MongoDB, the primary consideration should not be raw speed, as both perform simple key-value reads in sub-millisecond times.⁴⁷ The decision hinges on workload topology.⁴⁷ If an application requires massive, distributed horizontal write scaling across a cluster, or features an entirely dynamic schema at a massive scale, MongoDB remains the optimal choice.⁴⁷ Conversely, if the system relies heavily on complex analytical queries, deep relational JOINS, and strict financial-grade ACID compliance—while only needing document flexibility for specific sub-components—PostgreSQL JSONB is the superior and safer default choice.⁴⁷

Key Takeaways

- **Partitions are unavoidable physical realities:** The CAP theorem is not a restaurant menu allowing developers to "pick 2 of 3." Rather, given that network hardware will inevitably fail, a distributed system must actively choose to sacrifice either strong Consistency or Availability.
- **PACELC dictates day-to-day database performance:** While CAP only addresses rare failure states, PACELC is the practical framework for architectural design. It forces developers to evaluate the constant trade-off between Latency and Consistency in the absence of partitions.
- **Consistency is a configurable spectrum:** Distributed consistency ranges from strict Linearizability (expensive coordination, perfectly real-time) to Eventual Consistency (highly performant but prone to temporary data anomalies). MongoDB uniquely allows granular control of this spectrum via session-level readConcern and writeConcern.
- **Document modeling is entirely application-driven:** Unlike normalized SQL schemas which are modeled around the data itself, MongoDB schemas are designed around data *access patterns*. Patterns like the Computed Pattern and Extended Reference pre-aggregate and denormalize data to save CPU cycles and drastically reduce read latency.
- **Respect the physical hardware constraints:** The 16 MB BSON limit dictates document

design. Utilizing patterns like the Outlier and Bucket Pattern prevents unbounded arrays from growing indefinitely, ensuring fast RAM access and stable storage engine performance.

- **The SQL/NoSQL divide is converging:** PostgreSQL's JSONB enables high-performance document flexibility within strict relational bounds via GIN indexing, while MongoDB has integrated traditional relational features like JOINS (\$lookup) and multi-document ACID transactions to support enterprise workloads.

Further Reading

To explore the primary sources and mathematical foundations discussed in this document, refer to the following resources:

- Brewer, E. A. (2000). *Towards Robust Distributed Systems*. PODC Keynote. [Link to concept](#)
- Gilbert, S., & Lynch, N. (2002). *Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services*. ACM SIGACT News. [Link to formal proof](#)
- Abadi, D. J. (2012). *Consistency Tradeoffs in Modern Distributed Database System Design: CAP is Only Part of the Story*. IEEE Computer. [Link to PACELC](#)
- Kleppmann, M. (2015). *Please stop calling databases CP or AP*. [Link to blog](#)
- MongoDB Documentation. *Data Modeling Design Patterns*. (<https://www.mongodb.com/docs/manual/data-modeling/design-patterns/>)

Works cited

1. CAP theorem - Wikipedia, accessed March 28, 2026, https://en.wikipedia.org/wiki/CAP_theorem
2. (PDF) Towards robust distributed systems - ResearchGate, accessed March 28, 2026, https://www.researchgate.net/publication/221343719_Towards_robust_distributed_systems
3. The CAP Theorem. The Bad, the Bad, & the Ugly — Dominik Tornow, accessed March 28, 2026, <https://www.dtornow.com/blog/the-cap-theorem/>
4. Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services Seth Gilbert* Nancy Lynch* Abs, accessed March 28, 2026, <https://users.ece.cmu.edu/~adrian/731-sp04/readings/GL-cap.pdf>
5. Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services - cs.Princeton, accessed March 28, 2026, <https://www.cs.princeton.edu/courses/archive/spr22/cos418/papers/cap.pdf>
6. An Illustrated Proof of the CAP Theorem, accessed March 28, 2026, https://mwhittaker.github.io/blog/an_illustrated_proof_of_the_cap_theorem/
7. CAP Twelve Years Later: How the “Rules” Have Changed - GitHub Pages, accessed March 28, 2026, <https://sfu-db.github.io/dbsystems/Papers/CAP-12years.pdf>
8. Please stop calling databases CP or AP - Martin Kleppmann, accessed March 28, 2026,

- <https://martin.kleppmann.com/2015/05/11/please-stop-calling-databases-cp-or-ap.html>
9. An evaluation of relational and NoSQL distributed databases on a low-power cluster - PMC, accessed March 28, 2026,
<https://pmc.ncbi.nlm.nih.gov/articles/PMC10035467/>
 10. What Is Linearizability - Dagster, accessed March 28, 2026,
<https://dagster.io/glossary/linearizability>
 11. PACELC design principle - Wikipedia, accessed March 28, 2026,
https://en.wikipedia.org/wiki/PACELC_design_principle
 12. Consistency Tradeoffs in Modern Distributed Database System Design: CAP is Only Part of the Story | Semantic Scholar, accessed March 28, 2026,
<https://www.semanticscholar.org/paper/Consistency-Tradeoffs-in-Modern-Distributed-System-Abadi/2dbe6e8a70c9c4340435cf09231f32fd7f49a1e1>
 13. PACELC Theorem in Distributed Databases - DEV Community, accessed March 28, 2026, <https://dev.to/devcorner/pacelc-theorem-in-distributed-databases-pfe>
 14. Beyond CAP: Unveiling the PACELC Theorem for Modern Systems - DEV Community, accessed March 28, 2026,
<https://dev.to/ashokan/beyond-cap-unveiling-the-pacelc-theorem-for-modern-systems-465j>
 15. What is the PACELC Theorem? Definition & FAQs | ScyllaDB, accessed March 28, 2026, <https://www.scylladb.com/glossary/pacelc-theorem/>
 16. PACELC theorem - System Design Space, accessed March 28, 2026,
<https://system-design.space/en/chapter/pacelc-theorem/>
 17. The PACELC Theorem: Why Every Database Designer Must Make Impossible Choices, accessed March 28, 2026,
<https://medium.com/@benq019/the-pacelc-theorem-why-every-database-designer-must-make-impossible-choices-0dd9e259d59a>
 18. Redis cluster specification | Docs, accessed March 28, 2026,
https://redis.io/docs/latest/operate/oss_and_stack/reference/cluster-spec/
 19. Understanding Distributed Databases CAP and PACELC Concepts | by Kent Stroker, accessed March 28, 2026,
https://medium.com/@kent.stroker_49253/understanding-distributed-databases-cap-and-pacelc-concepts-d1a774de0ebe
 20. What is CAP Theorem? Definition & FAQs - ScyllaDB, accessed March 28, 2026,
<https://www.scylladb.com/glossary/cap-theorem/>
 21. What are the different types of consistency models? - Educative.io, accessed March 28, 2026,
<https://www.educative.io/blog/what-are-the-different-types-of-consistency-models>
 22. Linearizability versus Serializability | Peter Bailis, accessed March 28, 2026,
<https://www.bailis.org/blog/linearizability-versus-serializability/>
 23. What are Consistency Models? Definition & FAQs | ScyllaDB, accessed March 28, 2026, <https://www.scylladb.com/glossary/consistency-models/>
 24. Linearizability: `Now` in Distributed Systems | by Shavin Anjitha | Medium, accessed March 28, 2026,

<https://shavinanjitha.medium.com/linearizability-now-in-distributed-systems-8f379ffcb985>

25. What is Linearizability in Distributed System? | by James Kwon - Medium, accessed March 28, 2026,
<https://hongilkwon.medium.com/what-is-linearizability-in-distributed-system-db8bca3d432d>
26. Key Consistency Models to Know for Parallel and Distributed Computing - Fiveable, accessed March 28, 2026,
<https://fiveable.me/lists/key-consistency-models>
27. Sequential Consistency versus Linearizability - MIT, accessed March 28, 2026,
<https://groups.csail.mit.edu/tds/papers/Attiya/SPAA91.pdf>
28. Causal Consistency and Read and Write Concerns - Database ..., accessed March 28, 2026,
<https://www.mongodb.com/docs/manual/core/causal-consistency-read-write-concerns/>
29. Read Isolation, Consistency, and Recency - Database Manual - MongoDB Docs, accessed March 28, 2026,
<https://www.mongodb.com/docs/manual/core/read-isolation-consistency-recency/>
30. Read Concern - Database Manual - MongoDB Docs, accessed March 28, 2026,
<https://www.mongodb.com/docs/manual/reference/read-concern/>
31. Cost Optimization with Optimal Document Size - MongoDB, accessed March 28, 2026,
<https://www.mongodb.com/company/blog/technical/cost-optimization-with-optimal-document-size>
32. Documents - Database Manual - MongoDB Docs, accessed March 28, 2026,
<https://www.mongodb.com/docs/manual/core/document/>
33. What will happen if document size more than 16mb in MongoDB? - Stack Overflow, accessed March 28, 2026,
<https://stackoverflow.com/questions/23270124/what-will-happen-if-document-size-more-than-16mb-in-mongodb>
34. A Comprehensive Guide To MongoDB Data Modeling Design Patterns - GeoPITS, accessed March 28, 2026,
<https://www.geopits.com/blog/mongodb-data-modeling-design-patterns>
35. Building with Patterns: The Attribute Pattern | MongoDB, accessed March 28, 2026,
<https://www.mongodb.com/blog/post/building-with-patterns-the-attribute-pattern>
36. Building With Patterns: The Bucket Pattern | MongoDB, accessed March 28, 2026,
<https://www.mongodb.com/blog/post/building-with-patterns-the-bucket-pattern>
37. Building With Patterns: A Summary | MongoDB, accessed March 28, 2026,
<https://www.mongodb.com/company/blog/building-with-patterns-a-summary>
38. Building with Patterns: The Computed Pattern | MongoDB, accessed March 28, 2026,
<https://www.mongodb.com/company/blog/building-with-patterns-the-computed>

[-pattern](#)

39. Building with Patterns: The Extended Reference Pattern | MongoDB, accessed March 28, 2026,
<https://www.mongodb.com/blog/post/building-with-patterns-the-extended-reference-pattern>
40. Maintain Different Schema Versions - Database Manual - MongoDB Docs, accessed March 28, 2026,
<https://www.mongodb.com/docs/manual/data-modeling/design-patterns/data-versioning/schema-versioning/>
41. Using MongoDB Schema Validation With the Schema Versioning Pattern - Medium, accessed March 28, 2026,
<https://medium.com/mongodb/using-mongodb-schema-validation-with-the-schema-versioning-pattern-f51ce63ff376>
42. Building with Patterns: The Schema Versioning Pattern | MongoDB, accessed March 28, 2026,
<https://www.mongodb.com/company/blog/building-with-patterns-the-schema-versioning-pattern>
43. MongoDB Limits and Thresholds - Database Manual, accessed March 28, 2026,
<https://www.mongodb.com/docs/manual/reference/limits/>
44. MongoDB Time-Series: An Exploration of the Benefits, Limitations and Best Practices, accessed March 28, 2026,
<https://medium.com/@pbacc.uk/mongodb-time-series-an-exploration-of-the-benefits-limitations-and-best-practices-d3bbd391eba2>
45. About Time Series Data - Database Manual - MongoDB Docs, accessed March 28, 2026,
<https://www.mongodb.com/docs/manual/core/timeseries/timeseries-bucketing/>
46. Time Series data in MongoDB. Native TSDB capabilities in MongoDB | by Rajesh Rajagopalan | PeerAI Engineering Blogs, accessed March 28, 2026,
<https://engineering.peerislands.io/time-series-data-in-mongodb-4b998020a097>
47. PostgreSQL vs MongoDB: Which Database Should You Use in 2026? - AI2sql, accessed March 28, 2026, <https://builder.ai2sql.io/blog/postgresql-vs-mongodb>
48. MongoDB equivalent for PostgreSQL JSONB operations - DEV ..., accessed March 28, 2026,
<https://dev.to/mongodb/mongodb-equivalent-for-postgresql-jsonb-operations-hfo>
49. PostgreSQL B-tree vs GIN Index Performance : r/Backend - Reddit, accessed March 28, 2026,
https://www.reddit.com/r/Backend/comments/1q7ax4s/postgresql_btree_vs_gin_index_performance/
50. Comparing MongoDB vs. PostgreSQL, accessed March 28, 2026,
<https://www.mongodb.com/resources/compare/mongodb-postgresql>